简易聊天系统项目文档

姓名：池松泽
学号：523031910367

2025 年 12 月 20 日

目录

1 概述 **2**
1.1 运行环境 2
1.2 编译工具 2
1.3 程序文件列表 2

2 实现功能 **3**
2.1 客户端功能 3
2.2 服务端功能 3
2.3 通信协议 3

3 函数说明 **4**
3.1 客户端 (Client) 4
3.2 服务端 (Server) 4
3.3 工具包 (Utils) 5

4 程序测试 **6**
4.1 客户端登录 6
4.2 聊天会话 8
4.3 广播与私聊 9
4.4 多机测试 10

5 大模型交互 **11**

6 遇到的问题及解决办法 **12**

7 体会与建议 **12**
7.1 实验体会 12
7.2 改进建议 12

1 概述

本项目是一个基于 Go 语言实现的简易聊天程序，采用 C/S（客户端/服务端）架构。客户端使用 Fyne 框架构建图形用户界面（GUI），服务端负责处理消息转发、用户管理和广播。

1.1 运行环境

- **操作系统**: Windows, Linux, macOS
- **编程语言**: Go 1.25+
- **GUI 框架**: Fyne v2.7.1
- **网络协议**: TCP

1.2 编译工具

- **Go Compiler**: 用于编译源代码生成可执行文件。

1.3 程序文件列表

表 1: 项目文件清单

路径	文件名	说明
`source/cmd/client/`	`main.go`	客户端入口，包含网络逻辑
`source/cmd/client/`	`gui.go`	客户端界面逻辑及 UI 事件处理
`source/cmd/server/`	`main.go`	服务端入口，包含并发处理与路由
`source/utils/`	`utils.go`	通用工具包，定义消息结构与编解码
`source/`	`go.mod`	Go 模块定义文件
`source/`	`go.sum`	依赖模块校验和文件

2 实现功能

2.1 客户端功能

1. **用户登录**: 用户输入服务器 IP、端口和用户名进行连接。
2. **实时消息展示**: 接收并展示私聊、广播和系统消息。
3. **私聊**: 指定目标用户名发送点对点消息。
4. **广播**: 向所有在线用户发送消息。
5. **查询在线用户**: 请求服务器返回当前在线的用户列表。
6. **错误处理**: 处理连接断开和登录失败（如用户名重复）的情况。

2.2 服务端功能

1. **并发连接处理**: 支持多个客户端同时连接。
2. **用户注册与管理**: 维护在线用户列表，防止用户名重复。
3. **消息路由**: 包括 Text（私聊）、Broadcast（广播）和 Ask（查询列表）。
4. **系统通知**: 用户上下线广播、消息送达确认（Accept）及错误提示。

2.3 通信协议

客户端与服务端之间通过 TCP 连接通信，消息格式为 **Base64 编码的 JSON 字符串**，以换行符 \n 分隔。

消息结构 (utils.Message):

```go
type Message struct {
    Type     string `json:"type"`     // 消息类型: login, text, broadcast,
        ask, error, accept
    Content  string `json:"content"`  // 消息内容
    Sender   string `json:"sender"`   // 发送者用户名
    Receiver string `json:"receiver"` // 接收者用户名 (私聊时使用)
}
```

3 函数说明

3.1 客户端 (Client)

cmd/client/main.go

- `NewChatClient() *ChatClient`: 初始化一个新的聊天客户端实例。
- `Connect(ip, port, username string) error`: 建立 TCP 连接，发送登录消息，并验证登录结果。如果成功，启动后台监听协程。
- `listen(scanner *bufio.Scanner)`: 后台协程，持续监听服务器发来的消息，解码后通过 `OnMessage` 回调通知 GUI 更新。
- `Send(msg utils.Message)`: 将消息对象编码并发送给服务器。

cmd/client/gui.go

- `NewGUI() *GUI`: 初始化 Fyne 应用和窗口。
- `Run()`: 启动 GUI 主循环，首先显示登录界面。
- `showLogin()`: 渲染登录表单，处理登录按钮点击事件，调用 `client.Connect`。
- `showChat()`: 渲染聊天主界面，包括聊天记录滚动区、消息输入框、发送/广播/查询按钮。
- `handleMessage(msg utils.Message)`: 根据收到的消息类型 (`text, broadcast, error, accept`) 更新聊天记录或弹出错误提示。
- `appendChat(msg string)`: 将格式化后的消息追加到聊天记录显示区域，并自动滚动到底部。

3.2 服务端 (Server)

cmd/server/main.go

- `NewServer(port string) *Server`: 初始化服务端,创建监听器和各类消息通道 (`register, unregister, broadcast`)。
- `Run()`: 主循环，使用 `select` 多路复用处理新用户注册、用户注销和全局消息广播。
- `acceptLoop()`: 循环接受 TCP 连接请求，为每个连接启动 handleClient 协程。
- `handleClient(conn net.Conn)`: 处理单个客户端的生命周期：读取登录消息 -> 注册用户 -> 循环读取客户端消息 -> 转发到 `broadcast` 通道 -> 断开连接时注销。
- `handleMessage(msg utils.Message)`: 核心业务逻辑，根据 `msg.Type` 处理私聊转发、广播分发和用户列表查询。

- `readLogin(conn net.Conn) (*Client, error)`: 读取连接的第一条消息，验证是否为 `login` 类型，并提取用户名。
- `clientWriter(conn net.Conn, ch <-chan utils.Message)`: 后台协程，监听用户的发送通道，将消息编码并写入 TCP 连接。

3.3 工具包 (Utils)

utils/utils.go

- `JSONToBase64(v interface) (string, error)`: 将 Go 结构体序列化为 JSON，再进行 Base64 编码，用于网络传输。
- `Base64ToJSON(b64 string, v interface) error`: 将接收到的 Base64 字符串解码并反序列化为 Go 结构体。

4 程序测试

4.1 客户端登录

用户在登录时需要填入用户名，选填服务器 IP 地址和服务器端口号。

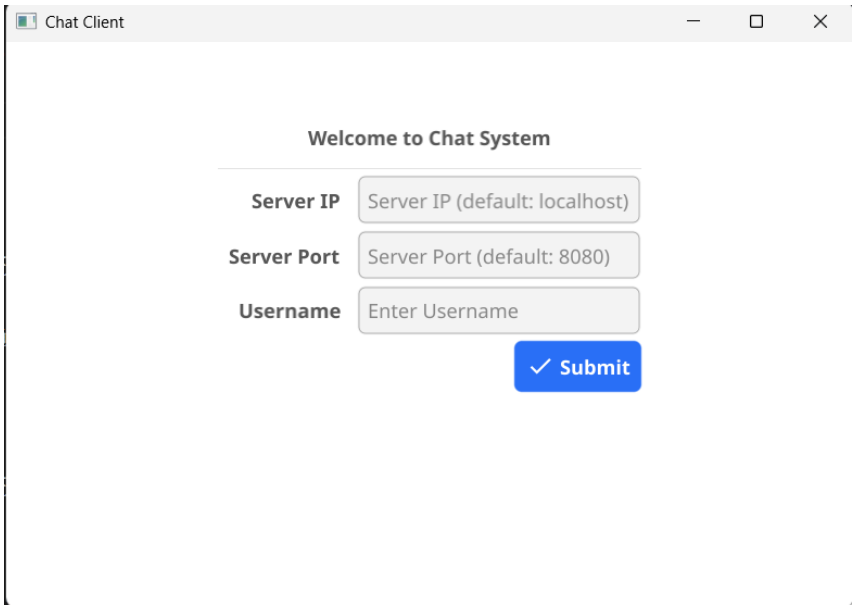


图 1: 登录界面

不填入用户名时会弹出报错弹窗：

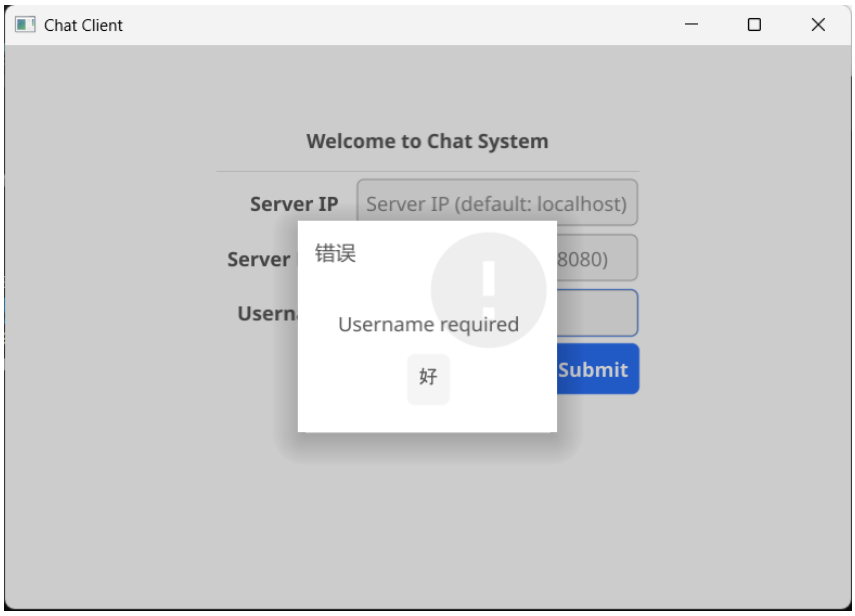


图 2: 用户名为空报错

当客户端尝试连接服务器时，会出现循环加载条：

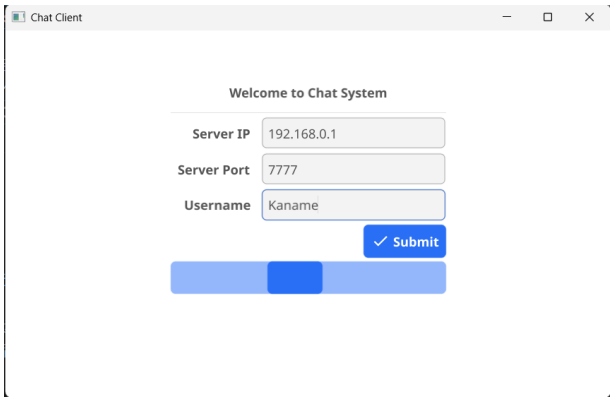


图 3: 连接加载中

当用户在登录中遇到连接问题时，例如尝试连接未运行服务器的 ip 端口或者使用了错误格式的 ip 端口等，会弹出错误信息弹窗。

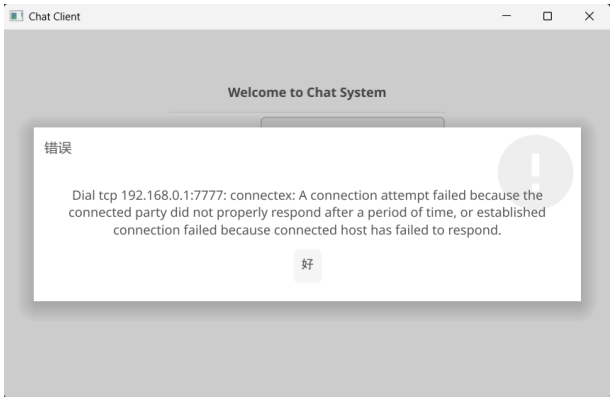


图 4: 连接错误提示

当使用重复用户名时，服务器会返回错误信息：

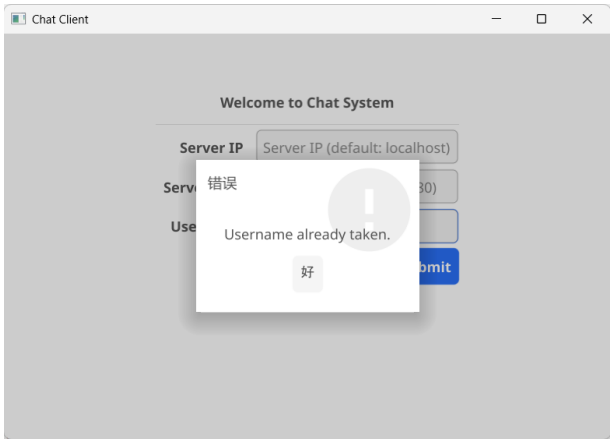
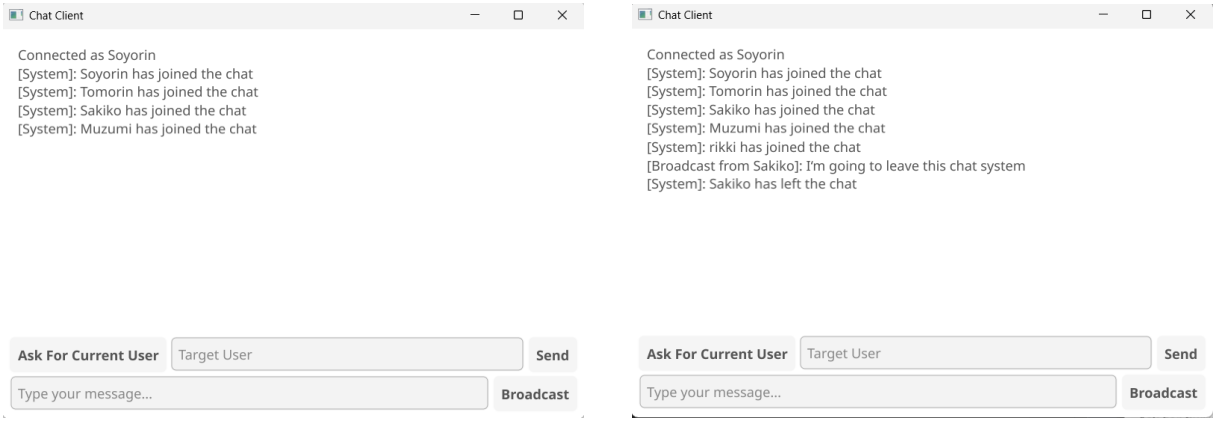


图 5: 用户名重复提示

4.2 聊天会话

用户登录和退出时，系统会向其他用户广播信息。



(a) 用户上线通知 (b) 用户下线通知

图 6: 系统广播通知

用户可以通过点击 `Ask For Current User` 按钮请求在线用户列表：

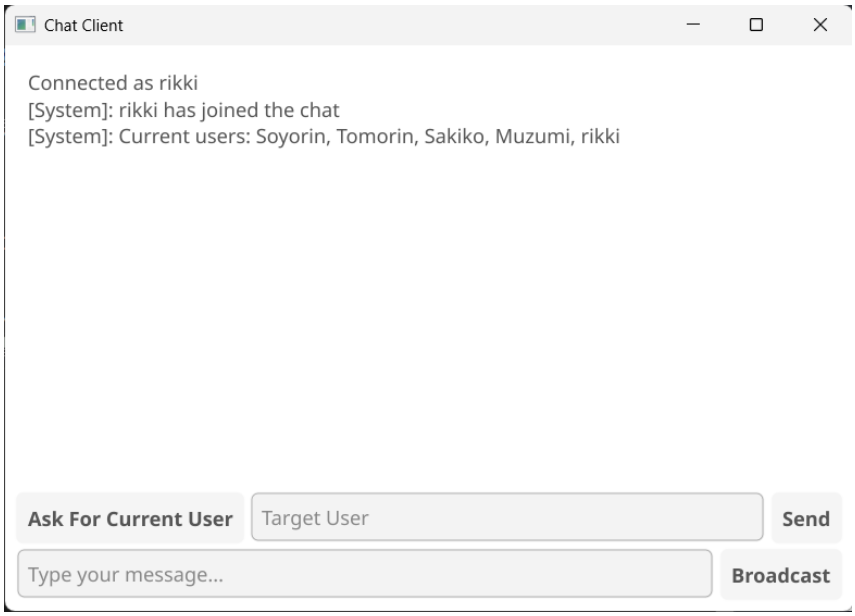


图 7: 查询在线用户结果

4.3 广播与私聊

用户可以通过 `Broadcast` 发送全员消息，或通过 `Send` 发送点对点私聊。

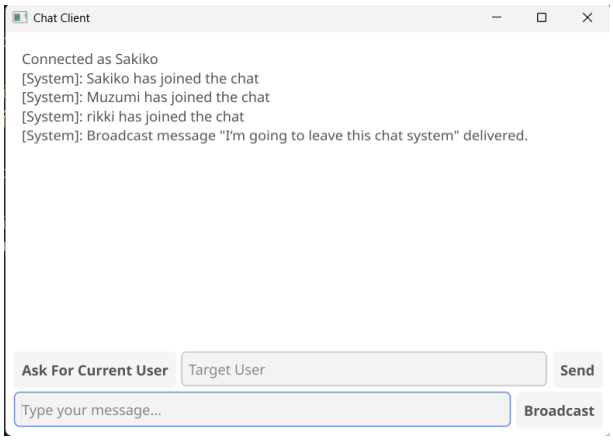


图 8: 发送广播消息

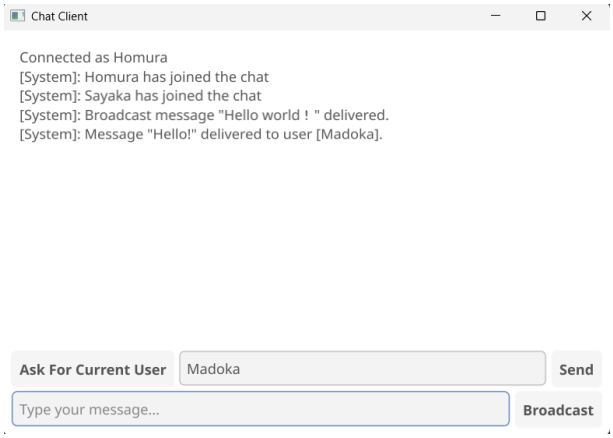


图 9: 发送私聊消息

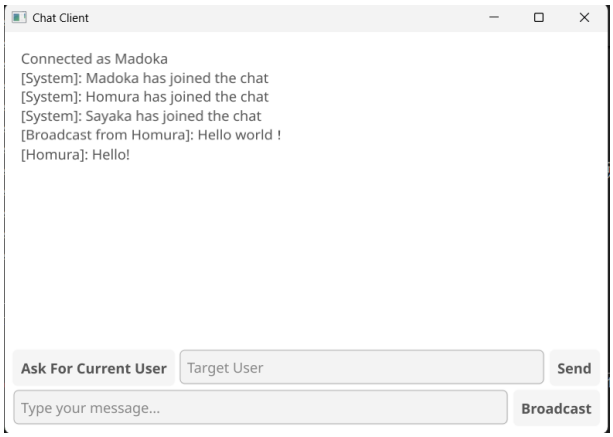


图 10: 接收消息

如果发送目标不存在或内容为空，系统会通过错误弹窗反馈：

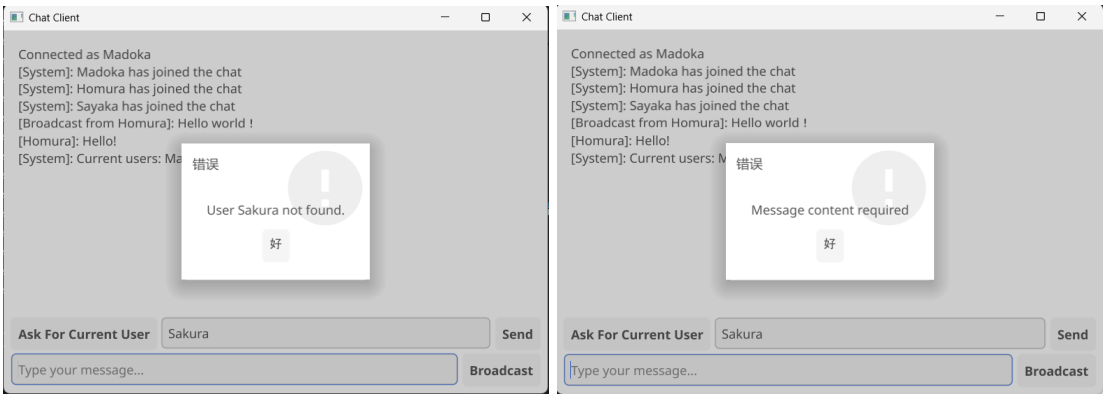
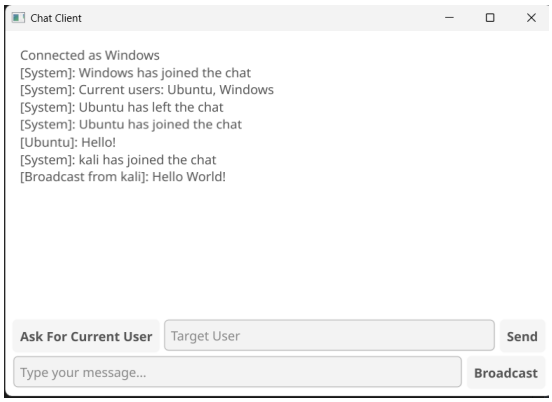


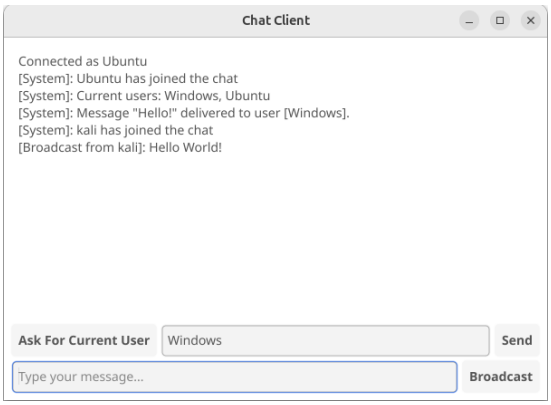
图 11: 发送错误提示（目标不存在/内容为空）

4.4 多机测试

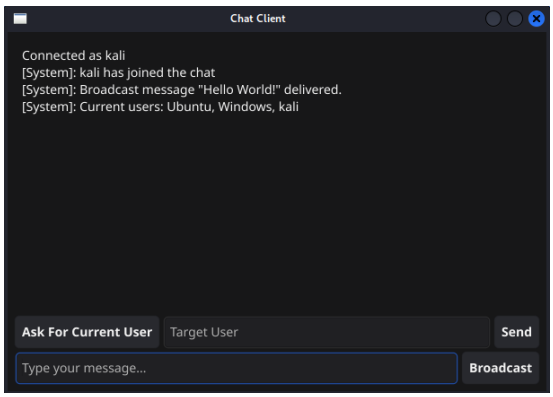
在 windows 本地以及 ubuntu、kali linux 两台虚拟机上启动客户端，在 windows 本地启动服务端监听。



(a) windows 客户端



(b) Ubuntu 客户端



(c) Kali Linux 客户端

图 12: 多机通信

5 大模型交互

该项目使用 copilot（接入 Gemini Pro 3 API）辅助完成，主要对 Fyne GUI 进行辅助编写以及对原代码重构优化。

图 13: 辅助编写 Fyne GUI

图 14: 重构优化源代码

6 遇到的问题及解决办法

- **如何处理多用户并发连接与消息分发？**: 服务端为每个连接启动独立的 Goroutine 进行读取，同时使用 Channel (register, unregister, broadcast) 将并发操作串行化，安全地管理 clients 映射表，避免了锁竞争和死锁风险。
- **如何处理客户端与服务端发送的信息中含有特殊字符的问题？**: 使用 json 标准化处理数据，并对数据进行 base64 编码，防止换行符等特殊字符破坏 TCP 消息边界。
- **如何避免网络阻塞导致 GUI 卡死？**: Fyne 的 UI 渲染运行在主线程。为了保持界面响应，所有的网络 IO 操作（如 Connect 和消息监听循环）都放入单独的 Goroutine 中执行，仅在需要更新界面时回调 UI 线程。

7 体会与建议

7.1 实验体会

通过本次实验，深入理解了 TCP 网络编程的基本原理，掌握了 Go 语言中 Goroutine 和 Channel 在高并发场景下的应用。同时，通过使用 Fyne 框架，体验了 Go 语言在 GUI 开发上的便捷性。在协议设计过程中，认识到了应用层协议定义（如消息类型、字段结构）对于系统健壮性的重要性。

7.2 改进建议

目前系统功能较为基础，未来可以从以下方面进行改进：

- **安全性**: 增加用户注册和密码登录功能，引入 TLS 加密传输。
- **功能扩展**: 支持离线消息存储、图片/文件传输、群组管理等功能。
- **用户体验**: 优化 UI 布局，增加消息气泡样式，支持快捷键操作。